

Universidade Federal de Uberlândia

Aula 14: Busca em árvore Monte Carlo II

AlphaZero

PUCT • Auto-jogo • Rede política-valor • AlphaTensor e AlphaDev • Raciocínio meta-nível

Prof. Marcelo Keese Albertini
Universidade Federal de Uberlândia
adaptado de Emma Brunskill, CS234 (Stanford)

2026



1. **Planejar agora, em vez de resolver tudo** — computação local por *uma* jogada; MCTS usa o orçamento.
2. **AlphaZero** — rollout vira rede de *política* e *valor*: PUCT, quatro fases, política da raiz.
3. **Auto-jogo** — busca vira dados de treino; adversário = cópia de si.
4. **A evidência** — arquitetura vs. busca vs. dados humanos; números do artigo.
5. **Além do tabuleiro** — AlphaTensor, AlphaDev, MuZero: descoberta como busca.
6. **Um olhar crítico** — UCT é o algoritmo certo para gastar simulações?

Onde isto se encaixa no curso

- ▶ Antes: política **para todo** o espaço de estados — iteração de valor, Q -learning, DQN, gradientes, PPO; $\pi : \mathcal{S} \rightarrow \mathcal{A}$.
- ▶ Hoje: dado s_t , computação para **uma** ação boa. Depois, repetir.
- ▶ As visões não competem: AlphaZero usa uma para melhorar a outra.

Intuição

Global = *amortizada*; local = *sob demanda*. $|S|$ astronômico: só a local viável — a global vira aproximação corrigida na hora.

SEÇÃO 1

Planejar agora, não resolver tudo

01

Busca Monte Carlo simples

Um passo de melhoria de política, e nada mais

Modelo (simulador) $\mathcal{M}$ + política de simulação π :

- ▶ Para cada ação $a \in \mathcal{A}$, simule K episódios a partir do estado real s_t :

$$\{s_t, a, r_{t+1}^k, s_{t+1}^k, \dots, s_T^k\}_{k=1}^K \sim \mathcal{M}, \pi$$

- ▶ Avalie cada ação pelo retorno médio:

$$Q(s_t, a) := \frac{1}{K} \sum_{k=1}^K G_t^k \xrightarrow{K \rightarrow \infty} Q^\pi(s_t, a)$$

- ▶ Escolha a ação real $a_t = \arg \max_{a \in \mathcal{A}} Q(s_t, a)$.

Atenção

Um passo de melhoria sobre π : guloso em Q^π , não π^* . π ruim $\Rightarrow$ estimativa ruim.

Podemos fazer melhor: a árvore expectimax

Com o modelo, dá para calcular $Q^*(s_t, a)$ *exatamente* — expandir a árvore completa a partir de s_t e retropropagar

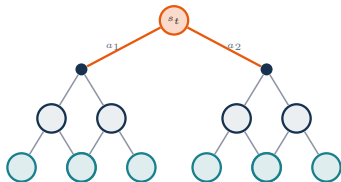
$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) \max_{a'} Q^*(s', a').$$

Definição — Busca à frente

Não é preciso resolver o MDP inteiro — apenas o *sub-MDP* que começa agora e vai até o horizonte H .

Atenção

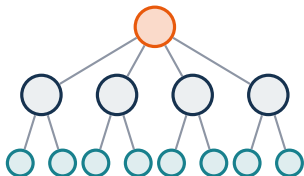
Árvore cresce como $(|S||\mathcal{A}|)^H$. No Go: fator de ramificação ≈ 250 , partida com ≈ 150 jogadas. Uma árvore de $250^{150} \approx 10^{360}$ folhas.



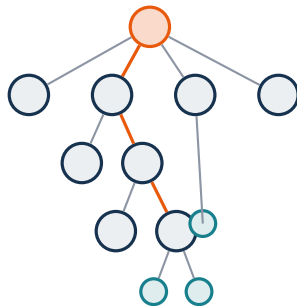
Busca em árvore Monte Carlo: crescer a árvore onde importa

Resposta ao $(|\mathcal{S}||\mathcal{A}|)^H$: não construir a árvore toda — árvore **assimétrica**, fundo só onde as simulações indicaram valer.

expectimax: uniforme e completa

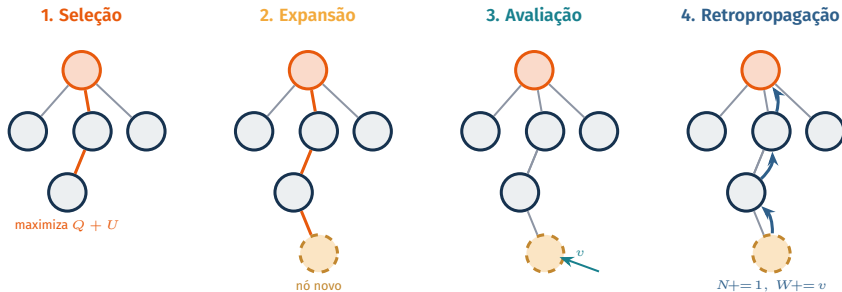


MCTS: seletiva e profunda onde promete



A árvore da direita é o *produto* do algoritmo, não sua entrada: cada simulação a torna informativa onde a decisão é difícil.

As quatro fases de uma simulação



Repita K vezes; ação real sai das estatísticas da raiz. **Tudo o desenho está em duas escolhas:** seleção (fase 1) e origem do v (fase 3).

UCT: cada nó da árvore é um bandit

Seleção: UCT (Kocsis e Szepesvári, 2006) — cada nó, ações como braços, cota superior de confiança.

Regra de seleção do UCT

$$Q(s, a, i) = \underbrace{\frac{1}{N(i, a)} \sum_{k=1}^{N(i, a)} G_k(i, a)}_{\text{média dos retornos simulados}} + \underbrace{c \sqrt{\frac{\log N(i)}{N(i, a)}}}_{\text{bônus de otimismo}}, \quad a_{ik} = \arg \max_a Q(s, a, i)$$

- ▶ $N(i, a)$: simulações por a em i ; $G_k(i, a)$: k -ésimo retorno dali.
- ▶ A política de simulação muda a cada episódio — a árvore se adapta ao já visto.
- ▶ $\bar{G}$: média sobre política não estacionária — última seção.

Por que a MCTS funciona tão bem

- ▶ **Busca seletiva pelo melhor primeiro.** Orçamento onde a decisão aperta, não onde a árvore é larga.
- ▶ **Avaliação dinâmica de estados.** Valor do nó construído pelas simulações que passam.
- ▶ **Amostragem quebra a maldição da dimensionalidade.** Custo = orçamento, não $|S|$.
- ▶ **Basta um modelo “caixa-preta”.** Nada de $P(s' | s, a)$ — só *amostrar* transições.
- ▶ **Barato, *anytime* e paralelizável.** Pare quando o relógio acabar; a resposta já serve.

Intuição

“Anytime”: iteração de valor nada entrega antes de convergir; MCTS entrega após a 1ª simulação. Xadrez, Go, tempo real: usável vs. inútil.

Atenção

Custo no orçamento: milhares de simulações por decisão — memória e pré-computação viram *tempo de decisão*.

Estudo de caso: o jogo de Go

- ▶ 2 500 anos de idade; o jogo de tabuleiro clássico mais difícil.
- ▶ Desafio histórico da IA — John McCarthy o citava como marco.
- ▶ Alfa-beta com avaliação manual **fracassou** no Go, ao contrário do xadrez.

	Xadrez	Go
Ramificação típica	≈ 35	≈ 250
Duração da partida	≈ 80	≈ 150
Posições legais	$\approx 10^{47}$	$\approx 10^{170}$
Avaliação manual boa?	sim	não

Teste sua compreensão

Jogar Go é aprender a decidir num mundo cuja **dinâmica** e cujo **modelo de recompensa** são desconhecidos? *Pergunta para discussão conduzida em aula.*

Estudo de caso: o jogo de Go

- ▶ 2 500 anos de idade; o jogo de tabuleiro clássico mais difícil.
- ▶ Desafio histórico da IA — John McCarthy o citava como marco.
- ▶ Alfa-beta com avaliação manual **fracassou** no Go, ao contrário do xadrez.

	Xadrez	Go
Ramificação típica	≈ 35	≈ 250
Duração da partida	≈ 80	≈ 150
Posições legais	$\approx 10^{47}$	$\approx 10^{170}$
Avaliação manual boa?	sim	não

Teste sua compreensão

Jogar Go é aprender a decidir num mundo cuja **dinâmica** e cujo **modelo de recompensa** são desconhecidos? *Pergunta para discussão conduzida em aula.*

Resposta

Não. Regras conhecidas e determinísticas; recompensa verificável. Falta é *usar* o modelo: $|S| \approx 10^{170}$, $H \approx 150$. AlphaZero = **planejamento** com heurísticas aprendidas.

SEÇÃO 2

AlphaZero: a busca guiada por uma rede

02

De AlphaGo a AlphaZero

Três sistemas, três graus de conhecimento humano embutido

	AlphaGo (2016)	AlphaGo Zero (2017)	AlphaZero (2018)
Partidas humanas	sim (rede supervisionada)	não	não
Redes	4 separadas	1 rede, 2 cabeças	1 rede, 2 cabeças
Arquitetura	convolucional	residual	residual
Avaliação da folha	rollout + rede de valor	só a rede de valor	só a rede de valor
Atributos manuais	vários	só as regras	só as regras
Domínios	Go	Go	Go, xadrez, shogi

Intuição

Sequência de *remoções* de muletas humanas; tirar a rollout — o “Monte Carlo” — *melhorou*. A Bayesiana já deslocava o peso para a rede de valor (Chen et al., 2018).

A rede: uma função, duas saídas

Uma rede residual f_θ recebe s (tabuleiro cru + histórico recente + vez) e devolve

$$f_\theta(s) = (p_\theta(s), v_\theta(s)), \quad p_\theta(s) \in \Delta(\mathcal{A}), \quad v_\theta(s) \in [-1, 1].$$

- ▶ $p_\theta(s)$ — **cabeça de política**: distribuição sobre as jogadas legais; *prior* por onde olhar.
- ▶ $v_\theta(s)$ — **cabeça de valor**: resultado final (+1 vitória, -1 derrota); substitui a rollout.

Definição — Corpo compartilhado

As duas cabeças partilham a torre residual. “Onde estou ganhando” e “o que jogar aqui” exigem as mesmas representações — reconhecer grupos vivos e mortos, influência, forma.

Uma passagem $\approx$ 100 rollouts

No AlphaGo de 2016, uma única passagem direta de v_θ atingiu erro quadrático médio comparável à média de 100 rollouts com a política de RL — a um custo de computação ordens de grandeza menor.

PUCT: a regra de seleção do AlphaZero

Por ação, no nó: $N(s, a)$, $W(s, a)$, $Q(s, a) = W/N$ e o prior $P(s, a) = \mathbf{p}_\theta(s)_a$. Ação simulada:

PUCT — *Predictor* + UCT

$$a_t = \arg \max_a \left[\underbrace{Q(s, a)}_{\text{exploração aprendida}} + \underbrace{c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}}_{U(s, a) : \text{bônus guiado pelo prior}} \right]$$

- ▶ N pequeno $\Rightarrow U$ domina: busca segue o **prior da rede**.
- ▶ Visitas crescem $\Rightarrow U \rightarrow 0$, Q assume: a busca **corrige** a rede por lookahead.
- ▶ c_{puct} regula a transferência de poder.

UCT e PUCT lado a lado

	UCT	PUCT (AlphaZero)
Bônus	$c \sqrt{\frac{\log N(s)}{N(s, a)}}$	$c_{\text{puct}} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)}$
Decaimento em $N(s, a)$	$N(s, a)^{-1/2}$	$N(s, a)^{-1}$ (mais agressivo)
Crescimento em $N(s)$	$\sqrt{\log N(s)}$	$\sqrt{N(s)}$ (muito mais forte)
Ação nunca visitada	bônus infinito	bônus finito, $\propto P(s, a)$
Origem	desigualdade de Hoeffding	heurística
Avaliação da folha	rollout até o fim	v_θ , sem rollout

Atenção

PUCT **não** é cota de confiança: prior baixo pode nunca ser visitado. Go: ≈ 250 jogadas, 1 600 simulações — viável, e exige o ruído de Dirichlet.

A simulação, do começo ao fim

1. Seleção. Da raiz, desça $\arg \max_a [Q + U]$ até s_L fora da árvore.

2. Expansão e avaliação. Uma chamada $f_\theta(s_L) = (p_\theta, v_\theta)$; nó com $N = W = Q = 0$, priors p_θ ; $v = v_\theta(s_L)$. **Não há rollout.**

3. Retropropagação. Para cada aresta (s, a) do caminho:

$$N += 1, \quad W += \pm v, \quad Q = W/N.$$

Sinal alterna com o jogador da vez.

Intuição

Rollout $\rightarrow$ rede = MC $\rightarrow$ TD. Rollout: não enviesado vs. política de rollout, variância enorme, viés vs. π^* . Rede: bootstrap, viés baixo que encolhe com o treino.

Definição — Min-max implícito

Como v é sempre “do ponto de vista de quem joga”, alternar o sinal na subida faz a árvore se comportar como uma árvore min-max — sem que “minimizar” apareça em lugar algum do código.

O PUCT em câmera lenta

Um exemplo mínimo que cabe no quadro

Raiz com três jogadas; a rede erra: prior alto em a_1 , a pior das razoáveis. Folhas: $v_\theta = (0,10; 0,55; -0,40)$; $c_{\text{puct}} = 1,5$.

k	$Q(s, a)$			$U(s, a)$			$Q + U$			escolhe
	a_1	a_2	a_3	a_1	a_2	a_3	a_1	a_2	a_3	
1	0,00	0,00	0,00	0,90	0,45	0,15	0,90	0,45	0,15	a_1
2	0,10	0,00	0,00	0,45	0,45	0,15	0,55	0,45	0,15	a_1
3	0,10	0,00	0,00	0,42	0,64	0,21	0,52	0,64	0,21	a_2
4	0,10	0,55	0,00	0,52	0,39	0,26	0,62	0,94	0,26	a_2
5	0,10	0,55	0,00	0,60	0,30	0,30	0,70	0,85	0,30	a_2
6	0,10	0,55	0,00	0,67	0,25	0,34	0,77	0,80	0,34	a_2
7	0,10	0,55	0,00	0,74	0,22	0,37	0,84	0,77	0,37	a_1
8	0,10	0,55	0,00	0,60	0,24	0,40	0,70	0,79	0,40	a_2
após 16 simulações: $N = (5, 11, 0)$ • a_3 só recebe sua primeira visita em $k = 23$										

O que a tabela ensina

- ▶ $k = 1-2$: **a rede manda**. $Q \equiv 0$; escolha = prior.
- ▶ $k = 3$: **a evidência entra**. a_1 devolve 0,10, abaixo do prior $\rightarrow$ busca migra.
- ▶ $k = 7$: **a busca volta a a_1** . $1 + N(s, a_2)$ cresceu; U confere.
- ▶ a_3 **fica de fora até $k = 23$** . Prior baixo + orçamento pequeno = jogada nunca examinada.

Atenção

Prior muito errado custa caro. $P = (0,95; 0,04; 0,01)$, mesmo v_θ :

simulações	$N(a_1)$	$N(a_2)$	recomenda
50	33	17	a_1 (errado)
200	44	156	a_2
800	89	711	a_2

A busca *corrige* a rede; preço = orçamento. Rede melhor $\Rightarrow$ menos simulações.

c_{puct} : quanto duvidar da própria rede

Mesmo exemplo, 400 simulações, variando só c_{puct} :

c_{puct}	$N(a_1)$	$N(a_2)$	$N(a_3)$	leitura
0,5	13	386	1	confia na rede; converge rápido, arrisca perder a jogada rara
1,0	25	373	2	
1,5	37	360	3	
3,0	71	324	5	<i>faixa usada na prática</i>
5,0	107	284	9	duvida da rede; gasta orçamento reexaminando o já descartado

Intuição

Não é cosmético: Bayesianas nos hiperparâmetros da MCTS rendeu **94 e 103 Elo** antes do Lee Sedol; 100 Elo $\approx$ 64% de vitórias.

Ruído de Dirichlet: garantir que tudo possa ser tentado

Prior abre a porta; se a de a_3 nunca abre, a rede errada não é descoberta. Em **auto-jogo** (não em partida oficial), perturba-se o prior *apenas na raiz*:

$$P(s_0, a) = (1 - \varepsilon) p_{\theta}(s_0)_a + \varepsilon \eta_a, \quad \eta \sim \text{Dir}(\alpha), \quad \varepsilon = 0,25.$$

Definição — Escolha de α

α é calibrado *inversamente* ao número típico de jogadas legais:

jogadas legais		α
Xadrez	≈ 35	0,30
Shogi	≈ 90	0,15
Go	≈ 250	0,03

α pequeno concentra o ruído em poucas jogadas sorteadas — exploração *profunda*, não diluição uniforme.

Intuição

Só na raiz: dá para tentar a estranha, descartável quando Q mostra que é ruim — exploração que a evidência veta, oposto do ϵ -guloso.

Atenção

Removido em avaliação e partidas oficiais: mecanismo de *coleta de dados*, não de jogo.

A política da raiz: contagens de visita, não valores

Terminadas as K simulações, a política exportada na raiz:

$$\pi(a \mid s_0) = \frac{N(s_0, a)^{1/\tau}}{\sum_b N(s_0, b)^{1/\tau}}.$$

No exemplo anterior, $N = (5, 11, 0)$:

τ	$\pi(a_1)$	$\pi(a_2)$	$\pi(a_3)$
1,00	0,312	0,688	0
0,50	0,171	0,829	0
0,25	0,041	0,959	0
$\rightarrow 0$	0	1	0

Auto-jogo: $\tau = 1$ nas 30 primeiras jogadas (diversidade), depois $\tau \rightarrow 0$.

Teste sua compreensão

Por que exportar as *contagens de visita* em vez do $\arg \max_a Q(s_0, a)$, que parece a estimativa mais direta do que é bom? *Pergunta para discussão conduzida em aula.*

A política da raiz: contagens de visita, não valores

Terminadas as K simulações, a política exportada na raiz:

$$\pi(a \mid s_0) = \frac{N(s_0, a)^{1/\tau}}{\sum_b N(s_0, b)^{1/\tau}}.$$

No exemplo anterior, $N = (5, 11, 0)$:

τ	$\pi(a_1)$	$\pi(a_2)$	$\pi(a_3)$
1,00	0,312	0,688	0
0,50	0,171	0,829	0
0,25	0,041	0,959	0
$\rightarrow 0$	0	1	0

Auto-jogo: $\tau = 1$ nas 30 primeiras jogadas (diversidade), depois $\tau \rightarrow 0$.

Teste sua compreensão

Por que exportar as *contagens de visita* em vez do $\arg \max_a Q(s_0, a)$, que parece a estimativa mais direta do que é bom? *Pergunta para discussão conduzida em aula.*

Resposta

N mais robusto que Q : visitada 3 vezes, ação pode ter Q alto por sorte. Contagens incorporam a confiança.

Por que a busca melhora a política

A afirmação central de todo o sistema:

A MCTS como operador de melhoria de política

A política da raiz π é, tipicamente, mais forte do que a política p_θ que guiou a busca. A busca é um **operador de melhoria**: consome p_θ e v_θ , gasta computação, devolve algo melhor.

De onde vem. $p_\theta(s_0)$ julga por padrões; π após olhar K continuações — informação que p_θ não tinha.

Consequência. Treinar p_θ para imitar π = passo de melhoria; repetir = iteração de política, busca no lugar do guloso.

Intuição

Deliberação treina intuição: busca = pensar devagar; rede = pensar rápido — *Expert Iteration* (Anthony, Tian e Barber, 2017).

Atenção

“Tipicamente” $\neq$ “sempre”: v_θ sistematicamente enviesado numa região $\rightarrow$ mais busca amplifica o erro.

O que o PUCT está realmente otimizando

Grill et al., ICML 2020

Contagens não são subproduto arbitrário: resolvem aproximadamente um **problema de otimização de política regularizada**:

$$\bar{\pi} \approx \arg \max_{y \in \Delta(\mathcal{A})} \left[\mathbf{q}^\top y - \lambda_N \text{KL}(\mathbf{p}_\theta \parallel y) \right], \quad \lambda_N = \frac{c_{\text{puct}} \sqrt{N}}{N + |\mathcal{A}|},$$

cuja solução é $\bar{\pi}_a = \lambda_N p_{\theta,a} / (\alpha - q_a)$, com α normalizando.

Verificando no exemplo do início ($P = (0,6; 0,3; 0,1)$, $q = (0,10; 0,55; -0,40)$, $c_{\text{puct}} = 1,5$):

	visitas empíricas $N_a / (N + \mathcal{A})$			previsão da forma fechada		
N	a_1	a_2	a_3	a_1	a_2	a_3
64	0,1940	0,7463	0,0149	0,2069	0,7755	0,0176
256	0,1120	0,8687	0,0077	0,1154	0,8752	0,0094
1600	0,0480	0,9464	0,0037	0,0486	0,9475	0,0039

Intuição

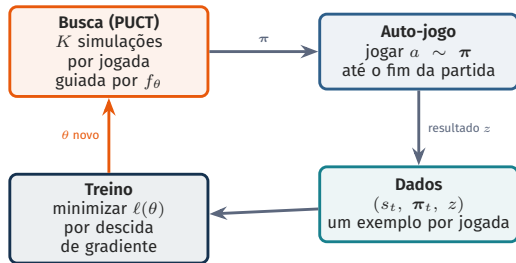
$\lambda_N \sim N^{-1/2} \rightarrow 0$: pouca busca $\Rightarrow$ perto do prior; muita $\Rightarrow \arg \max$ de q . A “proximidade regularizada” do TRPO/PPO, agora *dentro* de um planejador.

SEÇÃO 3

Auto-jogo: o laço que fabrica os próprios dados

03

O laço completo



Nenhum humano no laço.
O único insumo externo são as **regras do jogo** — usadas pelo simulador para gerar sucessores e detectar o fim da partida.

Partidas por iteração	25 000
Simulações por jogada	1 600 ($\approx 0,4$ s por busca)
Temperatura	$\tau = 1$ nas 30 primeiras jogadas, depois $\tau \rightarrow 0$
Desistência	automática, mantendo falsos positivos abaixo de 5%

Os dois alvos, e por que só um é escasso

Cada jogada gera (s_t, π_t, z) , com $z \in \{-1, 0, +1\}$ o resultado *final* visto de s_t .

A perda

$$\ell(\theta) = \underbrace{(z - v_\theta(s))^2}_{\text{regressão do valor}} - \underbrace{\pi^\top \log p_\theta(s)}_{\text{entropia cruzada: imitar a busca}} + \underbrace{c \|\theta\|^2}_{\text{regularização}}$$

- ▶ **Alvo de política: denso.** 200 jogadas $\rightarrow$ 200 alvos π_t ricos (distribuições, não rótulos duros); melhoria destilada.
- ▶ **Alvo de valor: escasso.** Um bit por partida, replicado em todas as posições; exemplos correlacionados, sinal fraco.

Atenção

“Go tem recompensa esparsa” = meia verdade: alvo de treino da política é denso — a busca fabrica supervisão em cada posição. Gradiente puro patinaria.

Auto-jogo é um currículo automático

- ▶ **O gargalo é só computação.** Sem anotador, especialista ou banco de partidas.
- ▶ **Adversário sempre à altura** — cópia de si; vitórias $\approx 50\%$ no treino todo.
- ▶ Adversário na medida = definição operacional de **currículo**: dificuldade sobe com a competência.

Teste sua compreensão

Em que isso ajuda o treino da política? E o que acontece com a densidade do sinal de recompensa? *Pergunta para discussão conduzida em aula.*

Auto-jogo é um currículo automático

- ▶ **O gargalo é só computação.** Sem anotador, especialista ou banco de partidas.
- ▶ **Adversário sempre à altura** — cópia de si; vitórias $\approx 50\%$ no treino todo.
- ▶ Adversário na medida = definição operacional de **currículo**: dificuldade sobe com a competência.

Teste sua compreensão

Em que isso ajuda o treino da política? E o que acontece com a densidade do sinal de recompensa? *Pergunta para discussão conduzida em aula.*

Resposta

Equilíbrio: z quase moeda honesta — entropia máxima, ≈ 1 bit por partida. Muito mais forte: $z \equiv -1$, nada a aprender; fraco: o mesmo, sinal trocado. Auto-jogo mantém a recompensa densa em informação.

Intuição

Por que RL funciona em jogos e sofre no real: jogos vêm com gerador de currículo embutido; robótica não.

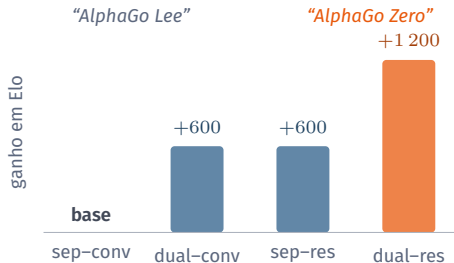
SEÇÃO 4

O que a evidência mostra

04

Quanto vem da arquitetura?

Quatro redes, *mesmo* conjunto fixo de partidas: política/valor *separadas* (sep) ou *conjuntas* (dual); *conv* ou *res*. Mesma busca, 5 s por jogada.

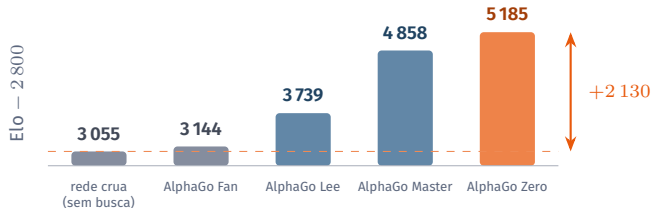


Intuição

Cada mudança vale sozinha mais de **600 Elo**; juntas, mais de 1200. Parte do “algoritmo” era engenharia de rede.

Quanto vem da busca?

Rede do AlphaGo Zero **sem nenhum lookahead** — $\arg \max_a p_\theta(s)_a$ — contra o sistema completo:



A busca vale $\approx 2\,100$ Elo

A mesma rede, com e sem MCTS, difere em mais de 2000 pontos — numa escala em que 200 pontos já são 75% de vitórias.

Intuição

Busca = **amplificador de computação em tempo de decisão** — a mesma alavanca do *test-time compute*.

Os dados humanos são necessários?

Mesmo laço de treino: rede iniciada por supervisionado sobre partidas humanas (KGS) vs. pesos aleatórios.

- ▶ Supervisionada: **mais rápida no início**, prevê jogadas humanas melhor — o tempo todo.
- ▶ *Tabula rasa*: ultrapassa em poucas horas, termina **mais forte**.
- ▶ Zero derrotou a versão do Lee Sedol **100 a 0**, 4 TPUs vs. 48.
- ▶ 3 dias: 4,9 milhões de partidas. 40 dias, 40 blocos: 29 milhões, Elo 5 185.

Atenção

Prever o humano $\neq$ jogar bem. Imitação carrega o teto do imitado; o auto-jogo não tem teto.

Intuição

Simulador perfeito + recompensa verificável é raro; sem eles (diálogo, medicina, direção), “dispensar humanos” *não* se transfere.

Um algoritmo, três jogos

Removeu o restante específico do Go (simetrias do tabuleiro, torneio entre gerações); mesmo código em xadrez, shogi e Go.

	AlphaZero	Stockfish 8
Paradigma de busca	MCTS guiada por rede	alfa-beta com poda
Posições por segundo	$\approx 6 \times 10^4$	$\approx 6 \times 10^7$
Conhecimento embutido	regras do xadrez	livro de aberturas, avaliação, finais
Partidas (Science 2018)	1 000 jogos: 155 vitórias, 6 derrotas, 839 empates	
Treino	700 000 passos; ultrapassou o Stockfish em ≈ 4 h	

Intuição

$10^3 \times$ menos posições examinadas, vitória: a rede não acelera a busca — escolhe *muito melhor* o que examinar.

Atenção

Condições contestadas (versão, hardware, tempo, sem livro de aberturas); superior até 10:1 em desvantagem de tempo. Informativo, não medição limpa.

SEÇÃO 5

Além do tabuleiro

05

A ideia que sobrevive à mudança de domínio

A receita, sem menção a jogos

Se um problema pode ser escrito como *sequência de escolhas* com um simulador que verifica o resultado, então uma rede que propõe candidatos mais uma busca que testa candidatos pode superar a heurística humana — mesmo quando o espaço de busca é astronômico.

Go / xadrez

estado: tabuleiro
ação: jogada
prêmio: vitória

AlphaTensor

estado: tensor residual
ação: tríade de rank 1
prêmio: rank baixo

AlphaDev

estado: programa parcial
ação: instrução
prêmio: correto e rápido

MuZero

estado: latente
ação: ação
modelo: **aprendido**

Só muda o simulador; busca, PUCT, auto-jogo e perda conjunta permanecem.

AlphaTensor: multiplicar matrizes mais rápido

Multiplicar $n \times n$ = decompor $\mathcal{T}_n$ em tríades de posto 1; *posto* = número de multiplicações escalares. Ação = tríade; episódio termina quando o tensor residual zera.

Problema	melhor conhecido	AlphaTensor	observação
2×2 (Strassen, 1969)	7	7	redescoberto do zero
3×3 (Laderman, 1976)	23	23	redescoberto do zero
4×4 sobre $\mathbb{Z}_2$	49	47	primeiro avanço desde 1969
4×5 por 5×5 (reais)	80	76	
5×5 (reais)	98	96	

Intuição

4×4 vale em $\mathbb{Z}_2$, não nos reais — distinção que a imprensa perdeu. Ainda assim, primeiro avanço em dois níveis em meio século.

Atenção

Penalizar o *tempo medido* na V100 rendeu 10–20% *naquele* hardware: otimalidade é relativa à máquina. Depois, Kauers e Moosbauer: 5×5 GF(2) a 95, busca clássica.

AlphaDev: escrever assembly mais rápido que o humano

AssemblyGame: estado = programa assembly construído; ação = instrução; recompensa = *correção* (testada contra entradas) + *latência*.

- ▶ Alvo: *sort3*, *sort4*, *sort5* — folhas de qualquer ordenação maior.
- ▶ Duas sequências novas (*swap*, *copy*): economizam uma instrução por aplicação.
- ▶ *VarSort4*: estrutura diferente, 29 instruções mais curto que o humano.
- ▶ Integrado à *libc++* do LLVM — primeira mudança na década e a primeira vinda de RL.

Ganhos medidos

Até **70%** mais rápido em sequências de cinco elementos; $\approx 1,7\%$ em sequências acima de 250 000 elementos. Também $\approx 30\%$ em funções de *hash* na faixa de 9 a 16 bytes.

Intuição

Rotinas otimizadas são *folhas*: melhora muito a folha, pouco o todo — mas chamadas trilhões de vezes por dia.

MuZero e o retorno do modelo desconhecido

AlphaZero exige simulador perfeito; MuZero **aprende** um modelo — só o suficiente para planejar.

Três funções aprendidas conjuntamente:

- ▶ **Representação** h : observação $\mapsto$ estado latente s^0 .
- ▶ **Dinâmica** g : $(s^k, a) \mapsto (s^{k+1}, r^k)$, no espaço latente.
- ▶ **Predição** f : $s^k \mapsto (p, v)$.

MCTS roda *inteiramente* no latente; nada reconstruir a observação — só política, valor e recompensa certos.

Intuição

Suficiente para decidir, não para prever — modelo orientado ao valor, não gerativo; prever pixels é caro e irrelevante.

Definição — Alcance

Mesmo algoritmo, mesmo desempenho em Go, xadrez e shogi — e estado da arte em Atari, onde as regras não são dadas.

SEÇÃO 6

Um olhar crítico: o UCT é o algoritmo certo?

06

A pergunta incômoda

UCT: cada nó = bandit; cota superior de confiança escolhe a ação a simular.

Teste sua compreensão

Por que isso é um pouco estranho? Lembre por que as cotas superiores de confiança eram uma boa ideia em bandits. Existe mesmo um problema de exploração *versus* aproveitamento **dentro** de episódios simulados? *Pergunta para discussão conduzida em aula.*

A pergunta incômoda

UCT: cada nó = bandit; cota superior de confiança escolhe a ação a simular.

Teste sua compreensão

Por que isso é um pouco estranho? Lembre por que as cotas superiores de confiança eram uma boa ideia em bandits. Existe mesmo um problema de exploração *versus* aproveitamento **dentro** de episódios simulados? *Pergunta para discussão conduzida em aula.*

Resposta

Bandit real: braço puxado *custa*; arrependimento cumulativo é o objetivo certo. Na simulação não se perde nada — só CPU; objetivo: **melhor decisão na raiz ao final do orçamento**. UCT resolve o errado.

Arrependimento cumulativo e arrependimento simples

Definição — Arrependimento cumulativo

$$R_n = \sum_{t=1}^n (\mu^* - \mu_{A_t})$$

O que se *pagou* ao longo do caminho. É o objetivo quando as ações são reais.

Definição — Arrependimento simples

$$r_n = \mu^* - \mu_{\hat{A}_n}$$

A qualidade da *recomendação final* $\hat{A}_n$. É o objetivo quando as ações são simuladas.

Bubeck, Munos e Stoltz (2009): há um conflito

Quanto mais um algoritmo minimiza o arrependimento *cumulativo*, pior fica seu arrependimento *simples* no pior caso. Alocação uniforme: $r_n = O(e^{-cn})$.

Algoritmo com $R_n = O(\log n)$, como o UCB: existe instância em que r_n decai apenas **polinomialmente** em n .

Intuição

Para R_n pequeno, para-se de amostrar o aparentemente ruim — exatamente o que traria certeza sobre o melhor.

O que se deveria fazer: raciocínio metanível

Pergunta certa não é “qual ação simular por mais recompensa simulada”, e sim:

O problema metanível

Qual é a próxima computação a executar, dado que ela custa tempo e que o benefício é a chance de mudar a decisão da raiz?

- ▶ Literalmente um MDP: estados = crenças sobre valores; ações = *computações* (Hay, Russell, Tolpin e Shimony, 2012).
- ▶ Só vale se puder **mudar** a jogada; refinar o último colocado é desperdício.
- ▶ BRUE, SHOT, *sequential halving* em árvores: superam o UCT em orçamento fixo.

Intuição

PUCT afasta-se do UCT na direção certa (por acaso histórico): decaimento $N(s, a)^{-1}$, crescimento $\sqrt{N(s)}$ — o que Grill et al. formalizam.

A parede da profundidade

Um experimento pequeno, um resultado desconfortável

Árvore-armadilha (espírito de Coquelin e Munos, 2007): em cada d , $parar = 0,9 - 0,4 d/D$; *seguir* até $D = 1,0$. Tentação no começo, recompensa no fim.

Prob. de a raiz escolher a ação ótima
(MCTS com rollout aleatória; melhor c por célula)

D	$K = 200$	1 000	5 000	20 000
4	0,85	1,00	1,00	1,00
6	0,00	0,20	1,00	1,00
8	0,00	0,00	0,00	0,00
10	0,00	0,00	0,00	0,00

Com $D = 10$, nem 50 000 bastam.

Atenção

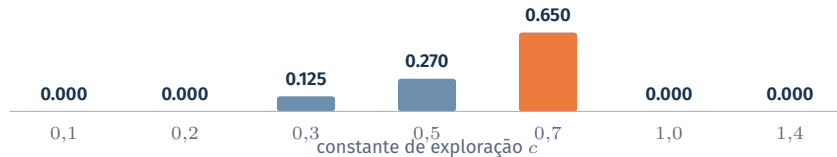
Duas causas: (i) rollout alcança a folha com prob. 2^{-D} ; (ii) UCT retropropaga a *média*, não o máximo — política mudou, viés se compõe ao subir.

Intuição

Coquelin e Munos: amostras *exponencial em exponencial* na profundidade; consistência assintótica verdadeira e inútil.

O ajuste de c não é secundário

Mesma árvore, $D = 6$, orçamento fixo de 2 000 simulações, variando só a constante de exploração:



Atenção

Curva **não monotônica nem suave**: $c = 0,7$ acerta 65%; $c = 1,0$ falha sempre. Pouca exploração não sai da armadilha; muita contamina as médias.

Intuição

Explica os 94/103 Elo só com Bayesiana nos hiperparâmetros; pergunte *quanto* ajuste houve — c pode valer mais que o algoritmo.

Então por que a MCTS funciona tão bem na prática?

1. **A rollout foi eliminada.** Causa (i) some: folha avaliada por v_θ , não por partida aleatória.
2. **O prior poda a largura.** ≈ 250 jogadas no Go $\rightarrow$ uma dúzia examinada; ramificação efetiva despenca.
3. **As árvores reais não são adversárias.** Em Go e xadrez, posições boas têm vizinhos bons — o gradiente existe.
4. **O laço de treino conserta a heurística.** Onde v_θ erra, o auto-jogo gera dados ali.

A leitura para levar

A MCTS moderna funciona apesar de o UCT ser questionável, não por causa dele: o peso saiu da regra de seleção e foi para a **qualidade do avaliador de folha**.

Atenção

Diagnóstico: suspeite primeiro do avaliador de folha e do prior; só depois da constante de exploração ou do orçamento.

SEÇÃO 7

Testes de compreensão

07

Quando a MCTS é uma boa escolha?

Teste sua compreensão

A MCTS decide qual ação tomar por busca em árvore, escolhendo ações que maximizam $Q(s, a)$ e *amostrando* sucessores. Em quais situações ela é boa escolha?

1. Horizonte curto, poucos estados e poucas ações.
2. Horizonte longo, **muitas ações** e poucos estados.
3. Horizonte longo, **muitos estados** e poucas ações.

Pergunta para discussão conduzida em aula.

Quando a MCTS é uma boa escolha?

Teste sua compreensão

A MCTS decide qual ação tomar por busca em árvore, escolhendo ações que maximizam $Q(s, a)$ e *amostrando* sucessores. Em quais situações ela é boa escolha?

1. Horizonte curto, poucos estados e poucas ações.
2. Horizonte longo, **muitas ações** e poucos estados.
3. Horizonte longo, **muitos estados** e poucas ações.

Pergunta para discussão conduzida em aula.

Resposta

Falso, falso, verdadeiro. (1) MDP exato: iteração de valor devolve π^* para todos os estados, mesmo preço. (2) A árvore ramifica *em cada ação*; poucos estados $\rightarrow$ programação dinâmica viável. (3) Amostragem substitui a soma sobre s' ; custo = orçamento, não $|S|$.

Tratar cada nó como um bandit

Teste sua compreensão

No UCT, cada nó é tratado como um bandit e uma cota superior de confiança sobre o valor futuro de cada ação decide o que simular em seguida. Quais afirmações são verdadeiras?

1. É útil porque prioriza ações que levam a boas recompensas adiante.
2. O UCB minimiza arrependimento; o UCT minimiza arrependimento *dentro* das rollouts da árvore — e, se é verdade, é uma boa ideia?

Pergunta para discussão conduzida em aula.

Tratar cada nó como um bandit

Teste sua compreensão

No UCT, cada nó é tratado como um bandit e uma cota superior de confiança sobre o valor futuro de cada ação decide o que simular em seguida. Quais afirmações são verdadeiras?

1. É útil porque prioriza ações que levam a boas recompensas adiante.
2. O UCB minimiza arrependimento; o UCT minimiza arrependimento *dentro* das rollouts da árvore — e, se é verdade, é uma boa ideia?

Pergunta para discussão conduzida em aula.

Resposta

As duas são verdadeiras — a segunda é o ponto. Arrependimento simulado **não é um custo**; objetivo = arrependimento *simples* na raiz — minimizar um prejudica o outro.

Revisão: para que servem as cotas superiores de confiança

Teste sua compreensão

Quais afirmações são verdadeiras?

1. Cotas superiores de confiança equilibram exploração e aproveitamento da informação já obtida.
2. Servem tanto em bandits quanto em processos de decisão de Markov.
3. Se o modelo de recompensa é conhecido, não há benefício em usá-las.

Pergunta para discussão conduzida em aula.

Revisão: para que servem as cotas superiores de confiança

Teste sua compreensão

Quais afirmações são verdadeiras?

1. Cotas superiores de confiança equilibram exploração e aproveitamento da informação já obtida.
2. Servem tanto em bandits quanto em processos de decisão de Markov.
3. Se o modelo de recompensa é conhecido, não há benefício em usá-las.

Pergunta para discussão conduzida em aula.

Resposta

Verdadeira. Verdadeira. Depende do cenário. Bandit: modelo de recompensa conhecido encerra o problema. **MDP:** não — com $P(s' | s, a)$ desconhecida, incerteza sobre *para onde* cada ação leva. Recompensa conhecida $\neq$ valor conhecido.

SEÇÃO 8

Síntese

08

O que levar desta aula

1. **Go: modelo conhecido, grande demais** — AlphaZero é planejamento com heurísticas aprendidas.
2. **Busca = operador de melhoria de política.** Treinar para imitar a busca = iteração de política.
3. **Auto-jogo = gerador de currículo.** Resultado incerto e, portanto, informativo.
4. **Alavanca principal: avaliador de folha.** Rollout → rede de valor = MC → TD (viés-variância).
5. **PUCT ≠ cota de confiança.** Regularização ao prior, enfraquece com $N^{-1/2}$.
6. **UCT resolve o problema errado.** Importa o arrependimento simples; MCTS moderna funciona apesar disso.
7. **Busca = computação em tempo de decisão.** Mesma rede vale 2 100 Elo a mais pensando antes.
8. **Cuidado ao extrapolar.** Simulador perfeito + recompensa verificável é raro; “dispensar humanos” não transfere.

Fontes primárias

- ▶ Silver et al., *Mastering the game of Go with deep neural networks and tree search*, Nature 529, 2016. (AlphaGo; a mistura rollout/valor.)
- ▶ Silver et al., *Mastering the game of Go without human knowledge*, Nature 550, 2017. (AlphaGo Zero; as ablações de arquitetura e busca.)
- ▶ Silver et al., *A general reinforcement learning algorithm that masters chess, shogi and Go through self-play*, Science 362, 2018.
- ▶ Schrittwieser et al., *Mastering Atari, Go, chess and shogi by planning with a learned model*, Nature 588, 2020. (MuZero.)
- ▶ Fawzi et al., *Discovering faster matrix multiplication algorithms with reinforcement learning*, Nature 610, 2022. (AlphaTensor.)
- ▶ Mankowitz et al., *Faster sorting algorithms discovered using deep reinforcement learning*, Nature 618, 2023. (AlphaDev.)

Fundamentos e crítica

- ▶ Kocsis e Szepesvári, *Bandit based Monte-Carlo planning*, ECML 2006. (UCT.)
- ▶ Coquelin e Munos, *Bandit algorithms for tree search*, UAI 2007. (As patologias.)
- ▶ Bubeck, Munos e Stoltz, *Pure exploration in multi-armed bandits problems*, ALT 2009. (Arrependimento simples versus cumulativo.)
- ▶ Hay, Russell, Tolpin e Shimony, *Selecting computations: theory and applications*, UAI 2012. (Raciocínio metanível.)
- ▶ Anthony, Tian e Barber, *Thinking fast and slow with deep learning and tree search*, NeurIPS 2017. (Expert Iteration.)
- ▶ Grill et al., *Monte-Carlo tree search as regularized policy optimization*, ICML 2020.
- ▶ Chen et al., *Bayesian optimization in AlphaGo*, arXiv:1812.06855, 2018.
- ▶ Browne et al., *A survey of Monte Carlo tree search methods*, IEEE TCIAIG 4(1), 2012. (Panorama.)